

Exemple d'affichage complet

Objectifs

- Synthétiser les éléments clés d'un chapitre.
- Illustrer avec un extrait de code concis.
- Donner 1 à 2 repères visuels (tableau, schéma).

Code Python (basique)

root@python:~\$

```
def somme(liste):  
    total = 0  
    for x in liste:  
        total += x  
    return total
```

```
print(somme([1, 2, 3])) # 6
```

Définitions

Algorithme : suite finie d'instructions permettant de résoudre un problème.

Complexité : quantité de ressources (temps, mémoire) consommées par un algorithme.

Idiomes Python

root@python:~\$

```
# Compréhension de liste  
evens = [x for x in range(10) if x % 2 == 0]
```

Slicing

```
s = "NSI"  
print(s[::-1]) # ISN
```

Tableau compact

Structure Mutable	Clés/Index
list	Oui Index entiers
tuple	Non Index entiers
dict	Oui Clés hachables

Schéma (image)



Les Chaînes de Caractères en Python

💡 Définition et Création

Une **chaîne de caractères** (ou **string**) est une séquence ordonnée et **immuable** de caractères. On la reconnaît par les guillemets simples `' '` ou doubles `" "` qui l'entourent. Une chaîne de caractères peut contenir des lettres, des chiffres, des symboles et des espaces.

```
root@python:~$  
  
# Création d'une chaîne de caractères vide  
ma_chaine = ""  
# Création d'une chaîne avec du texte  
salut = "Bonjour le monde"  
# Obtenir la longueur d'une chaîne avec len()  
longueur = len(salut)  
print(f"La chaîne 'salut' contient {longueur} caractères.")  
# Affiche "La chaîne 'salutation' contient 16 caractères."
```

⚙️ Saisie et Traitement

Saisie avec input()

La fonction `input()` permet de demander à l'utilisateur de saisir une chaîne de caractères. Le programme s'arrête en attendant que l'utilisateur entre du texte et appuie sur Entrée. La valeur saisie est renvoyée sous forme de chaîne de caractères.

```
root@python:~$  
  
nom = input("Entrez votre nom : ")  
print(f"Bonjour, {nom} !")  
# Si l'utilisateur tape "Alice"  
# la console affichera : "Bonjour Alice !"
```

📍 Indexation et Accès aux éléments

Chaque caractère d'une chaîne possède une position, appelée **index**. En Python, ⚠️ l'**indexation commence à 0**.

On peut aussi utiliser des index négatifs pour accéder aux caractères à partir de la fin de la chaîne. L'index -1 correspond au dernier caractère, -2 à l'avant-dernier, et ainsi de suite.

```
root@python:~$  
  
mot = "Python"  
# Indexation positive  
print(mot[0]) # Affiche "P"  
# Indexation négative  
print(mot[-1]) # Affiche "n"  
# Le slicing : Extraire une partie de la chaîne [début:fin]  
# ⚠️ La fin est exclue !  
print(mot[1:4]) # Affiche "yth"  
print(mot[2:]) # Affiche "thon"
```

📄 Parcourir une chaîne

La boucle **for** est l'outil principal pour parcourir une chaîne de caractères.

1. Par caractère :

C'est la méthode la plus simple et la plus courante.

```
root@python:~$
```


💡 Définition et Création

Un **dictionnaire** est une collection de données **non ordonnée** et **modifiable**. Contrairement aux listes qui utilisent des index numériques, un dictionnaire stocke des données sous forme de paires **clé-valeur**. Chaque clé est unique et sert d'identifiant pour accéder à sa valeur. Un dictionnaire est reconnaissable par les accolades `{}` qui l'entourent.

```
root@python:~$
```

```
# Création d'un dictionnaire vide
mon_dictionnaire = {}
# Création d'un dictionnaire avec des éléments
personne = {"nom": "Alice", "age": 25, "ville": "Paris"}
# Obtenir le nombre d'éléments avec len()
nombre_elements = len(personne)
print(f"Le dictionnaire 'personne' contient {nombre_elements} éléments.")
# Affiche "Le dictionnaire 'personne' contient 3 éléments."
```

🔑 Accès aux éléments

Vous accédez à la valeur d'un dictionnaire en utilisant sa clé. ⚠️ Si la clé n'existe pas, une erreur est générée.

```
root@python:~$
```

```
etudiants = {
    "id_1": "Jean",
    "id_2": "Marie",
    "id_3": "Paul"
}
# Accès par la clé
print(etudiants["id_1"]) # Affiche "Jean"
# Utilisation de la méthode .get() avec une valeur par défaut
print(etudiants.get("id_2", "Non trouvé")) # Affiche "Marie"
print(etudiants.get("id_4", "Non trouvé")) # Affiche "Non"
# Utilisation de la méthode .get() sans valeur par défaut
print(etudiants.get("id_2")) # Affiche "Marie"
# Remarque : La méthode .get() ne génère pas d'erreur si la
```

⚙️ Modification du dictionnaire

Ajouter et modifier des éléments

Vous pouvez ajouter une nouvelle paire clé-valeur ou modifier la valeur d'une clé existante en utilisant la syntaxe de l'indexation.

```
root@python:~$
```

```
voiture = {"marque": "Ford", "modele": "Mustang"}
# Ajout d'un nouvel élément
voiture["annee"] = 1964
print(voiture) # {'marque': 'Ford', 'modele': 'Mustang', 'annee': 1964}
```

📖 Parcourir un dictionnaire

La boucle `for` est l'outil principal pour parcourir un dictionnaire.

1. Par les clés :

C'est la méthode la plus simple et la plus courante.

```
root@python:~$
```

```
profil = {"nom": "Alex", "age": 30, "poste": "Designer"}
for cle in profil:
    print(cle)
```


💡 Définition et Création

Une liste est une collection de données ordonnée et modifiable, on parle de **Mutable**. Elle est reconnaissable par les crochets `[]` qui l'entourent. On peut y stocker des éléments de types différents (nombres, chaînes de caractères, booléens, etc.).

```
root@python:~$
```

```
# Création une liste vide
ma_liste = []
# Création une liste avec des éléments
fruits = ["pomme", "banane", "orange"]
# Obtenir la longueur d'une liste avec len()
longueur = len(fruits)
print(f"La liste 'fruits' contient {longueur} éléments.")
# Affiche "La liste 'fruits' contient 3 éléments."
```

⚙️ Modification de la liste

Ajouter des éléments

append(element) : ajoute un élément à la fin de la liste.

insert(index, element) : insère un élément à un index précis.

extend(iterable) : ajoute tous les éléments d'un autre objet (comme une autre liste) à la fin.

```
root@python:~$
```

```
jours = ["lundi", "mardi"]
jours.append("jeudi")
print(jours) # ['lundi', 'mardi', 'jeudi']
jours.insert(2, "mercredi")
```

📍 Indexation et Accès aux éléments

Chaque élément d'une liste possède une position, appelée index. En Python, ⚠️ l'indexation commence à 0.

On peut aussi utiliser des index négatifs pour accéder aux éléments à partir de la fin de la liste. L'index -1 correspond au dernier élément, -2 à l'avant-dernier, et ainsi de suite.

```
root@python:~$
```

```
animaux = ["chien", "chat", "oiseau", "poisson"]
# Indexation positive
print(animaux[0]) # Affiche "chien"
# Indexation négative
print(animaux[-1]) # Affiche "poisson"
# Le slicing : Extraire une partie de la liste [début:fin]
# ⚠️ La fin est exclue !
print(animaux[1:3]) # Affiche ['chat', 'oiseau']
print(animaux[2:]) # Affiche ['oiseau', 'poisson']
```

📖 Parcourir une liste

La boucle for est l'outil principal pour parcourir une liste.

1. Par élément :

C'est la méthode la plus simple et la plus courante. On parcourt directement chaque valeur de la liste.

```
root@python:~$
```


💡 Définition et Création

Une **variable** est un espace de stockage nommé qui contient une valeur. En Python, vous n'avez pas besoin de déclarer le type de la variable; l'interpréteur le détermine automatiquement en fonction de la valeur que vous lui attribuez.

📁 Types de Données de Base

Python dispose de plusieurs types de données fondamentaux :

- **Nombres entiers (int)** : Pour les nombres sans décimales. Exemple : `age = 25`.
- **Nombres flottants (float)** : Pour les nombres à virgule. Exemple : `taille = 1.75`.
- **Chaînes de caractères (str)** : Pour le texte. Elles sont délimitées par des guillemets simples ou doubles. Exemple : `nom = "Alice"`.
- **Booléens (bool)** : Représentent une valeur de vérité, qui peut être soit **True** (vrai) soit **False** (faux). Exemple : `est_majeur = True`.

📝 Règles pour nommer les variables

Les noms de variables doivent suivre des règles précises pour être valides en Python. S'ils ne les respectent pas, cela entraînera une erreur de syntaxe.

- Un nom de variable doit commencer par une lettre (a-z, A-Z) ou un trait de soulignement (`_`). Il ne peut pas commencer par un chiffre.
- Un nom de variable ne peut contenir que des caractères alphanumériques (a-z, A-Z, 0-9) et des traits de soulignement (`_`).
- Les noms de variables sont sensibles à la casse (`age`, `Age` et `AGE` sont des variables différentes).
- Un nom de variable ne peut pas être un mot-clé Python réservé (comme `if`, `for`, `class`, etc.).

root@python:~\$

```
# Exemples de noms de variables valides
mon_age = 30
_nom_utilisateur = "admin"
vitesse_1 = 120
```

🔧 Les Opérateurs Fondamentaux

2. Les Opérateurs Arithmétiques ➗

1. L'Opérateur d'Affectation (=)